

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA14

COURSE NAME: Compiler

Design

COURSE OUTCOME COVERED

***CO4:** Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 10%

Duration :60 Mins

In Class

QUESTIONS: 10

Total Marks:50

Annexure A

DATA INTERPRETATION

Code of Conduct:

I Ismail M (192524128) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Ismail M (192524128)

Q1. Three-Address Code Analysis

Given:

$t1 = a * b$

$t2 = t1 + c$

$t3 = t2 - d$

(i) Sequence of computations:

1. $t1$ stores the result of multiplying a and b : $t1 = a \times b$.

2. $t2$ adds c to the previously computed value $t1$: $t2 = (a \times b) + c$.

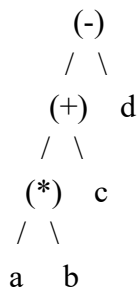
3. $t3$ subtracts d from $t2$: $t3 = ((a \times b) + c) - d$.

Therefore, the final result represented by $t3$ is $(a \times b) + c - d$.

(ii) Equivalent expression tree:

The root operator is $-$. Its left child is $+$ and its right child is d . The $+$ node has $a \times b$ as its left child and c as its right child.

Expression tree:



(iii) Role of temporary variables:

Temporary variables store intermediate results so that each computed value can be reused by subsequent instructions. Here, $t1$ stores $a \times b$ and $t2$ stores $(a \times b) + c$. This avoids recomputing earlier expressions and makes the intermediate representation simple, systematic, and efficient for later optimization.

Q2. Symbol Table and Memory Allocation

Given:

$p : \text{int} = 4 \text{ bytes}$

$q : \text{float} = 8 \text{ bytes}$

$r : \text{char} = 1 \text{ byte}$

$s : \text{double} = 8 \text{ bytes}$

(i) Memory allocation:

Each variable is assigned storage according to its declared type size:

- p requires 4 bytes.

- q requires 8 bytes.
- r requires 1 byte.
- s requires 8 bytes.

The compiler records the type and size in the symbol table and uses this information for storage allocation.

(ii) Total memory requirement:

Total = 4 + 8 + 1 + 8 = 21 bytes.

Thus, the raw storage requirement is 21 bytes.

(iii) Alignment and padding:

Memory alignment may require variables to begin at addresses suitable for their data types. Padding bytes can therefore be inserted between variables or at the end of the structure. For the individual variables listed here, the raw total is 21 bytes; the actual layout can be larger if alignment constraints require padding. The exact padded size depends on the target architecture and alignment rules.

Q3. Common Subexpression Elimination

Given:

$t1 = x + y$

$t2 = t1 * z$

$t3 = x + y$

$t4 = t3 - w$

(i) Step-by-step computation:

1. Compute $x + y$ and store it in $t1$.
2. Multiply $t1$ by z and store the result in $t2$.
3. Compute $x + y$ again and store it in $t3$.
4. Subtract w from $t3$ and store the result in $t4$.

(ii) Common subexpression:

The expression $x + y$ occurs twice:

- First occurrence: $t1 = x + y$
- Second occurrence: $t3 = x + y$

The second computation is redundant because x and y have not changed.

(iii) Optimized intermediate code:

$t1 = x + y$

$t2 = t1 * z$

$t4 = t1 - w$

The statement $t3 = x + y$ is eliminated and $t1$ is reused. This reduces the number of arithmetic operations and improves execution efficiency.

Q4. Short-Circuit Evaluation of OR

Given:

if ($a < b \parallel c > d$)

Intermediate code:

if $a < b$ goto L1

if $c > d$ goto L1

goto L2

L1: ...

(i) Control flow:

The first condition $a < b$ is evaluated. If it is true, control immediately jumps to L1, because the complete OR expression is already true. If $a < b$ is false, the second condition $c > d$ is evaluated. If it is true, control also goes to L1. If both conditions are false, control goes to L2.

(ii) Short-circuit evaluation:

For logical OR, the second condition is evaluated only when the first condition is false.

Thus:

- $a < b = \text{true} \rightarrow$ jump directly to L1.
- $a < b = \text{false} \rightarrow$ evaluate $c > d$.

This avoids unnecessary evaluation.

(iii) Execution examples:

1. $a=2, b=5, c=1, d=3$:

$a < b$ is true $\rightarrow$ L1 is executed; $c > d$ is not evaluated.

2. $a=5, b=2, c=4, d=3$:

$a < b$ is false; $c > d$ is true $\rightarrow$ L1 is executed.

3. $a=5, b=2, c=2, d=3$:

both conditions are false $\rightarrow$ L2 is executed.

Q5. Backpatching

Given:

120: if $p > q$ goto _

121: goto _

truelist = {120}

falselist = {121}

(i) Backpatching:

Backpatching is the process of filling unresolved jump targets after the target labels or addresses become known. Instruction 120 belongs to the truelist because it is executed when $p > q$ is true. Instruction 121 belongs to the falselist because it is executed when the condition is false.

(ii) Target labels:

Let the true branch target be L1 and the false branch target be L2. Then:

120: if $p > q$ goto L1

121: goto L2

The exact numeric target addresses are determined when the surrounding intermediate code is generated.

(iii) Control-flow resolution:

The truelist identifies instructions that must jump to the true destination, while the falselist identifies instructions that must jump to the false destination. Backpatching replaces the blank targets with the appropriate labels, producing complete control flow.

Q6. Three-Address Code for an Expression

Given:

$\text{result} = a * b + c / d$

(i) Three-address code:

$t1 = a * b$

$t2 = c / d$

$t3 = t1 + t2$

$\text{result} = t3$

(ii) Order of evaluation:

Multiplication and division have higher precedence than addition. Therefore, $a \times b$ and $c \div d$ are evaluated first. Their results are then added:

$\text{result} = (a \times b) + (c \div d)$

(iii) Role of temporary variables:

t1 stores $a \times b$, t2 stores $c \div d$, and t3 stores the sum of these intermediate results.

Temporary variables break a complex expression into simple three-address instructions, making the code easier to optimize and translate into machine-level instructions.

Q7. Switch Statement Translation

Given:

```
switch(choice) {  
    case 1: OPEN;  
    case 2: SAVE;  
    case 3: EXIT;  
}
```

(i) Intermediate code using conditional jumps:

if choice == 1 goto L1

if choice == 2 goto L2

if choice == 3 goto L3

goto L4

L1: OPEN

goto L4

L2: SAVE

goto L4

L3: EXIT

goto L4

L4: ...

This representation assumes that each case is intended as a separate branch; explicit jumps to the common exit prevent unintended fall-through.

(ii) Execution:

- If choice = 1, control jumps to L1 and OPEN is executed.
- If choice = 2, control jumps to L2 and SAVE is executed.
- If choice = 3, control jumps to L3 and EXIT is executed.
- If no case matches, control goes to L4.

(iii) Jump-table optimization:

For a dense range of case values, the compiler can create a jump table containing the target address for each case. The value of choice is used as an index into the table, allowing direct branching to the corresponding case instead of performing several sequential comparisons. This can make switch execution faster for many cases.

Q8. Procedure Call and Activation Record

Given:

total = compute(x, y)

Intermediate representation:

param x

param y

call compute, 2

(i) Parameter passing:

The caller places the arguments x and y into the parameter-passing area or appropriate registers/locations according to the calling convention. The instructions param x and param y specify the arguments, and call compute, 2 transfers control to compute with two parameters.

(ii) Activation record:

An activation record, or stack frame, is the block of memory associated with a particular procedure invocation. It can contain parameters, return address, saved registers, local

variables, temporaries, and control/link information. It provides the runtime environment needed while compute executes.

(iii) Return value:

After compute finishes, its result is returned according to the calling convention, commonly through a designated return-value register or a specified memory location. The caller receives this value and assigns it to total.

Q9. AND Condition and Backpatching

Given:

if (m > n && p < q)

Intermediate code:

if m > n goto L1

goto L3

L1: if p < q goto L2

goto L3

(i) Control flow:

The first condition $m > n$ is evaluated. If it is false, control immediately goes to L3, making the AND expression false. If it is true, control goes to L1, where $p < q$ is evaluated. If $p < q$ is true, control goes to L2; otherwise, control goes to L3. Therefore, L2 represents the true outcome and L3 represents the false outcome.

(ii) True and false lists:

During Boolean expression generation, unresolved jumps are stored in lists. For an AND expression, the true list of the first condition is connected to the beginning of the second condition because the second condition must also be true. False exits from either condition lead to the overall false list. Thus, the first condition's true target is L1, while false targets eventually reach L3.

(iii) Backpatching:

Backpatching fills unresolved jump targets after the required labels are known. The pending true jump from $m > n$ is patched to L1, the true jump from $p < q$ is patched to L2, and false paths are patched to L3. This completes the control-flow graph.

Q10. Data Dependencies and Data Flow

Given:

t1 = p * q

t2 = t1 + r

t3 = t2 * s

(i) Data dependencies:

- t1 depends on p and q.

- t2 depends on t1 and r.
- t3 depends on t2 and s.

Therefore, there is a dependency chain:

$p, q \rightarrow t1 \rightarrow t2 \rightarrow t3$

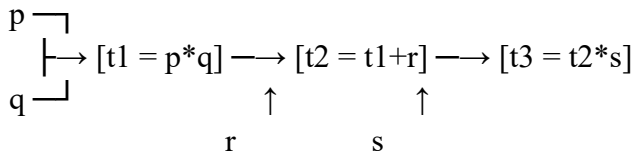
and r contributes to t2, while s contributes to t3.

(ii) Reuse of computed values:

The result of $p \times q$ is stored in t1 and reused by the next statement. Similarly, t2 stores $t1 + r$ and is reused by the final multiplication. No intermediate result needs to be recomputed.

(iii) Data-flow representation and optimization:

A simple data-flow representation is:



The code already avoids repeated computation. Possible optimizations include constant folding if any operands become constants, dead-code elimination if a temporary is never used, copy propagation where applicable, and machine-level instruction scheduling while preserving the dependencies.

Note: Answers are prepared directly from the questions and intermediate-code examples in the uploaded Assessment Tool 3 document. Where the document leaves target labels unspecified, symbolic labels (L1, L2, etc.) are used rather than inventing numeric addresses.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand